Option 6. Sequencing jobs through a common production line

Decision owner and action

A production planner chooses the order in which jobs enter a line of eight stages that every job visits in the same order. The tool chooses one sequence of all jobs, used at every stage, and reports stage-level start times.

Why this matters

Flow lines, e.g., printed circuit board assembly or furniture finishing, move every job through the same stages, and on many such lines jobs cannot pass one another, so the order chosen at the first stage holds at every stage. A job that is quick at the first stage but slow at a later stage blocks everything behind it, and the resulting waiting builds up downstream. Because jobs are released over time and carry due dates of different importance, the planner needs a single sequence that keeps weighted lateness low without letting work pile up between stages.

Files in the data package

File	Contents
`jobs.csv`	One row per job with release time, due date, and weight.
`stages.csv`	The stages and their order. Hand-checkable and edge cases have four stages, and all other cases have eight.
`processing_times.csv`	One row per job-stage pair with a positive processing time.

Required constraints

1. Every job is processed once at every stage, in stage order, without interruption.
2. The job sequence is identical at every stage.
3. A job starts the first stage no earlier than its release and starts each later stage no earlier than its completion at the previous stage.
4. Each stage processes one job at a time. Waiting space between stages is unlimited.

Required output

A `sequence` list (job identifiers in the common order) and a `schedule` list with one entry per job-stage pair containing `job_id`, `stage_id`, `start`, and `completion`.

Measures

The primary measure is total weighted tardiness, where a job's completion is its completion at the last stage. The secondary measure is total waiting between stages (the sum over jobs and later stages of the start at a stage minus the completion at the previous stage), and every comparison also reports computation time.

Rule to beat

Earliest due date sequence. Sort jobs by due date (ties by job identifier) and use that order at every stage. Each operation starts at the later of the job's completion at the previous stage (or its release, at the first stage) and the stage's completion of the previous job in the sequence.

Method directions and reading

Method 1 is the rule to beat. A researched method may adapt the insertion heuristic of Nawaz, Enscore, and Ham (1983, *Omega*) to weighted tardiness, use an iterated greedy search (Ruiz and Stützle, 2007, *European Journal of Operational Research*), or draw on the review of tardiness heuristics by Vallada, Ruiz, and Minella (2008, *Computers and Operations Research*).

Stress test

The uncertain input is processing times. The fixed-plan execution rule keeps the planned sequence and starts every operation as early as the realized processing times allow.

Scope boundary

The required model has one machine per stage, a common sequence, and unlimited waiting space. Different routes and alternative machines belong to option 2, and limited buffers are outside the required scope.

Checked example

Applying the rule to beat to the hand-checkable case gives a total weighted tardiness of 281 weighted minutes. The decisions and every recomputed measure appear in `worked-example/` in the data ZIP, with full precision in `metrics.json`. Reproducing this value to three decimals is a useful first test of a team's data loading, checker, and rule.

Where the rest of the project is defined

The project guide (`team-project-guide.pdf`) defines what every decision tool must do, the milestones, the grading rubric, the contents of this data package, and the exact rules the instructor's checker applies. This brief adds only what is specific to option 6.